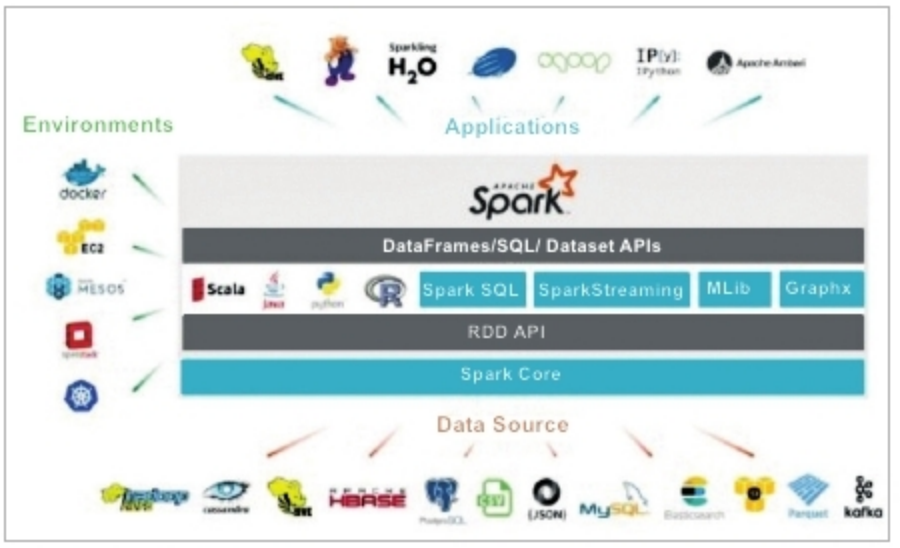

Figure 1: Spark ecosystem

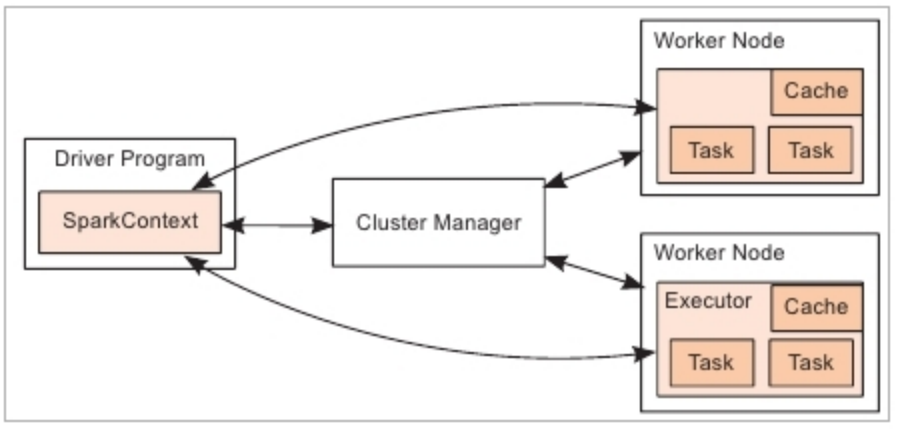

Figure 2: Spark cluster mode

scaling, and management of containerised applications). The Kubernetes scheduler is currently experimental.

How Spark works in cluster mode

Spark orchestrates its operations through the driver program, which is the process running the *main()* function of the application and creating the SparkContext. When the driver program is run, the Spark framework initialises the executor processes on the cluster hosts that process the data. The driver program must be network-addressable from the worker nodes. An executor is a process launched for an application on a worker node that runs tasks and keeps data in memory or disk storage. Each application has its own executors, and each driver and executor process in a cluster is a different Java Virtual Machine (JVM).

Here's what occurs when we submit a Spark application to a cluster:

1. The driver is launched and invokes the main method in the Spark application.
2. The driver requests resources from the cluster manager to launch executors.
3. The cluster manager launches executors on behalf of the driver program.
4. The driver runs the application. Based on the transformations and actions in the application, the driver sends tasks to the executors.
5. Tasks are run on executors to compute and save results.
6. If dynamic allocation is enabled, after the executors are idle for a specified period, they are released.

When the driver's main method exits or calls *SparkContext.stop*, it terminates any outstanding executors and releases resources from the cluster manager.

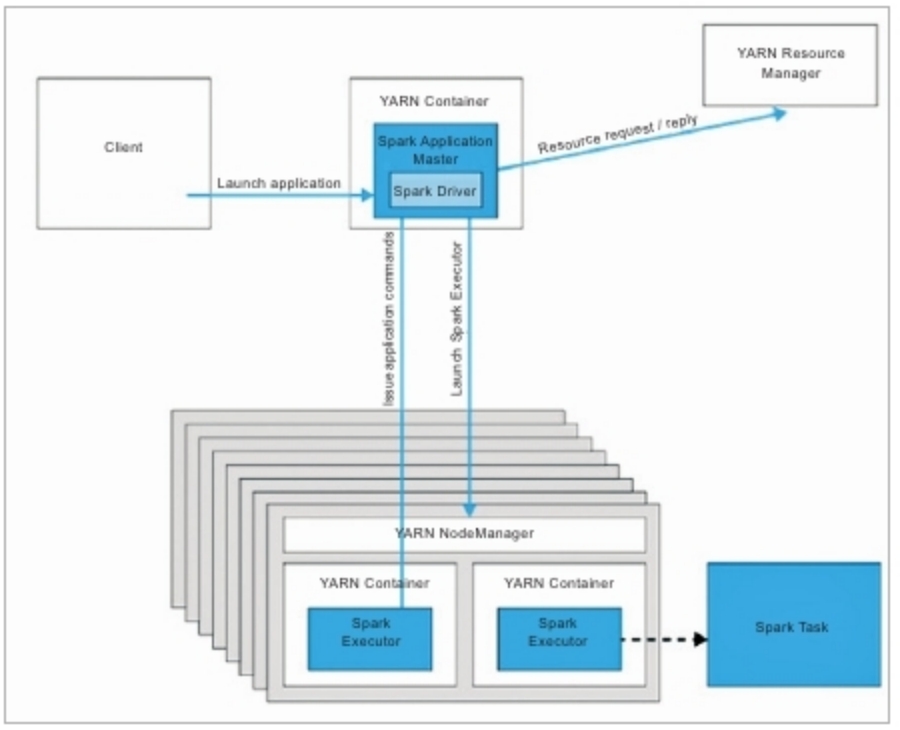

Figure 3: Spark in YARN cluster mode

Mode	YARN Client Mode	YARN Cluster Mode
Driver runs in	Client	ApplicationMaster
Requests resources	ApplicationMaster	ApplicationMaster
Starts executor processes	YARN NodeManager	YARN NodeManager
Persistent services	YARN ResourceManager and NodeManagers	YARN ResourceManager and NodeManagers
Supports Spark Shell	Yes	No

Figure 4: Summary of Spark in the YARN client and cluster modes

Now let us focus on running Spark applications on YARN.

The advantages of Spark on YARN

Listed below are reasons why Spark works best with YARN rather than other cluster managers.

1. We can dynamically share and centrally configure the same pool of cluster resources among all frameworks that run on YARN.
2. We can use all the features of YARN schedulers (the capacity scheduler, fair scheduler or FIFO scheduler) for categorising, isolating and prioritising workloads.
3. We can choose the number of executors to use; in contrast, Spark Standalone requires each application to run an executor on every host in the cluster.

Also, Spark can run against Kerberos-enabled Hadoop clusters and use secure authentication between its processes.

Deploying modes in YARN

In YARN, each application instance has an ApplicationMaster process, which is the first container started for that application. The application is responsible for requesting resources from the ResourceManager. Once the resources are allocated, the application instructs NodeManagers to start containers on its behalf. ApplicationMasters eliminate the need for an active client—the process starting the application can terminate and coordination continues from a process managed by YARN running on the cluster.